

Beyond Relational: Semantic-Aware Multi-Modal Analytics with LLM-Native Query Optimization

Anonymous Author(s)

Abstract

Multi-modal analytical processing has the potential to transform applications in e-commerce, healthcare, entertainment, and beyond. However, real-world adoption remains elusive due to the limited ability of traditional relational query operators to capture query semantics. The emergence of foundation models, particularly the large language models (LLMs), opens up new opportunities to develop flexible, semantic-aware data analytics systems that transcend the relational paradigm.

We present Nirvana, a multi-modal data analytics framework that incorporates programmable semantic operators while leveraging both logical and physical query optimization strategies, tailored for LLM-driven semantic query processing. Nirvana addresses two key challenges. **First**, it features an agentic logical optimizer that uses natural language-specified transformation rules and random-walk-based search to explore vast spaces of semantically equivalent query plans — far beyond the capabilities of conventional optimizers. **Second**, it introduces a cost-aware physical optimizer that selects the most effective LLM backend for each operator using a novel improvement-score metric. To further enhance efficiency, Nirvana incorporates computation reuse and evaluation pushdown techniques guided by model capability hypotheses. Experimental evaluations on three real-world benchmarks demonstrate that Nirvana is able to reduce end-to-end runtime by 10%–85% and reduces system processing costs by 76% on average, outperforming state-of-the-art systems at both efficiency and scalability.

ACM Reference Format:

Anonymous Author(s). 2018. Beyond Relational: Semantic-Aware Multi-Modal Analytics with LLM-Native Query Optimization. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Semantics-aware multi-modal data analytics has long been an attractive vision, with the potential to power important applications across domains such as e-commerce, healthcare, and entertainment. However, despite its promise, reliable deployment in production environments is still lacking. One major impediment is the rigidity of existing analytical processing systems, whose semantics-unaware operators often struggle to support the querying of unstructured

and heterogeneous data. Fortunately, the emergence of foundation models offers new opportunities, with Large Language Models (LLMs) demonstrating remarkable general-purpose reasoning capabilities, such as document understanding and question answering. This opens up new possibilities: the realization of flexible, semantic-aware multi-modal data analytics systems that hold the potential to fundamentally transform analytics by leveraging the complementary strengths of foundation models and large-scale data systems. On the one hand, LLMs bring semantic understanding across modalities — including text, images, and audio — into the core of data analytics. On the other hand, system engineering enables these models to be deployed with greater efficiency and transparency.

As a type of compound AI system, multi-modal semantic analytical processing systems can transcend monolithic models by being envisioned to offer the following appealing features:

- (1) **High-Quality & Versatile Data Analytics.** These systems are designed to support a wide range of analytics tasks, including descriptive, predictive, and prescriptive analytics, while delivering high-quality results that are robust and production-ready.
- (2) **Efficiency & Scalability.** Like relational database systems, semantics-aware multi-modal analytics systems must be capable of processing large-scale, data-intensive workloads efficiently and cost-effectively, scaling to tens of millions of records and beyond.
- (3) **Automatic Optimization.** Inspired by traditional relational query optimization, these systems should provide transparent, end-to-end optimization of semantic analytical workflows. Users should be able to specify *what* they want to achieve, without worrying about *how* to achieve it.

EXAMPLE 1 (PROMPT VS. COMPOUND SYSTEM). *Figure 1 illustrates two approaches to answering the query, “Compute the average total box office gross of all movies that Matt Damon stars in and whose IMDb rating is higher than 8.5,” over a multi-modal table, where the cast must be extracted from movie posters. The first approach (upper part) constructs a single prompt by concatenating the query with the full table contents and submits it directly to an LLM. In realistic scenarios — with tables containing hundreds or thousands of rows — this method fails because the prompt’s token length exceeds the model’s maximum context window, causing truncation or outright rejection. The alternative approach leverages a semantic analytical processing system that exposes a set of programmable operators powered by LLMs. The system decomposes the original query into a sequence of atomic semantic tasks — for example, extracting the cast from posters, filtering out movies with IMDb ratings below 8.5 or without Matt Damon in the cast, etc. Each task is executed on appropriately sized batches of records, ensuring that no single LLM invocation exceeds the context limit. Additionally, the system employs various system-level optimizations, such as parallel execution, to make the analytics more efficient and cost-effective.*

The recent surge of interest in semantic analytical processing has given rise to two complementary research strands. First, several

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

Table with structured and unstructured fields

Title	Overview	Gross	Rating	Poster
The Godfather	An organized crime dynasty's...	\$134,966,411	9.2	
The Dark Knight	When the menace known as the Joker wreaks...	\$534,858,444	9	
...	...	...	...	...

* Table has thousands of data records.

Approach 1: Prompting an LLM

Given a table regarding movies, please answer the following question:

##Question

What is the average total box office gross of all movies that Matt Damon stars in and whose rating is higher than 8.5? The cast is extract from the movie poster.

##Table Content

[Title	Overview	Gross	Rating	Poster]
 [The Godfather|An organized crime...|\$134,944,411|9.2|godfather_poster.png]
 [The Dark Knight|When the menace...|\$534,858,444|9|dark_knight_poster.png]

Exceed the max context length!



long context due to the size of the table

Question in natural language

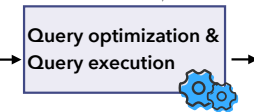
What is the average total box office gross of all movies that Matt Damon stars in and whose rating is higher than 8.5? The cast is extract from the movie poster.



Query written with semantic operators

```
filter("The rating is higher than 8.5")
map("Extract the list of the cast from the poster")
filter("Matt Damon is in the cast")
reduce("Compute the average total box gross movies")
```

Powered by LLMs



~\$174M

Approach 2: Using a data analytics system

Figure 1: Example of performing data analytics on a multi-modal movie table by (1) directly prompting an LLM or (2) using a data analytical processing system.

efforts integrate AI-powered operators into existing SQL engines. For example, ThalamusDB [11] introduces semantic filtering backed by pretrained models into DuckDB. CAESURA [26] augments traditional SQL operators (e.g., joins and aggregations) with semantic ones (e.g., TextQA and VisualQA) for question answering over text and images. While these extensions unlock new functionality, they exhibit low compatibility with the query optimizers in existing systems. Second, a growing body of studies target ground-up, purpose-built semantic analytical systems [2, 15, 19, 22, 27]. These systems expose declarative, natural-language-parameterized transformations — such as the ‘filter’ and ‘map’ operators used in Figure 1 — alongside traditional relational operators. Notably, they incorporate system-level optimizations: for example, Palimpsest [15] adapts the Cascades optimization framework to generate semantic query execution plans that strike a Pareto-optimal balance among quality, latency, and cost. Despite this progress, fundamental questions remain unanswered. Classical cost estimation falls short when the selectivity and performance of operators depend on LLM inference behavior. Moreover, *analysis quality* emerges as a first-class optimization objective due to the probabilistic nature and occasional hallucinations of foundation models. *Beyond simply extending SQL or reusing relational optimizers, what new optimization paradigms can unlock the full promise of multi-modal semantic processing?*

We present Nirvana, a novel multi-modal analytical processing framework that offers programmable semantic operator interfaces and introduces new strategies for system-level optimization. Rather than sidelining existing efforts on semantic processing, the goal of Nirvana is to present complementary optimization opportunities for semantic processing that existing systems overlook. In addition to implementing semantic operators that enable versatile data analytics, Nirvana contributes two new optimization designs, driven by the following key questions:

Question 1: What makes semantic query optimization different from traditional query optimization? Unlike traditional relational operators, **semantic operators** invoke LLM inference to

process multi-modal data, which is expensive and dominates the processing cost due to the high computational cost associated with LLM inference. Therefore, the prime object of query optimization is to reduce the number of LLM calls. Borrowing from relational DBMSs, Nirvana adopts transformation-based logical plan optimization (semantic query rewriting), employing traditional rules like predicate pushdown and new rules tailored to semantic analytics, e.g., operator fusion and non-LLM replacement. These transformations require the optimizer to understand query semantics that existing query optimizers cannot handle. LLMs are a silver bullet for semantic query optimization, as they have been shown to be capable of solving complex tasks such as workflow orchestration [9, 23, 30, 33]. Nirvana employs an agentic logical plan optimizer that leverages LLMs to explore semantically equivalent but more efficient plans using a random walk-based search framework. However, we note that, in relational databases, query optimization overhead is negligible compared to query execution (e.g., milliseconds for terabytes of data according to the TPC-H benchmark [25]), while LLM calls can take seconds to complete. Given the high cost of LLMs, **is it worth using LLMs for query optimization?** Our answer is yes. Unlike the query optimizations in relational databases that is based on data statics, *semantic query optimization can benefit markedly from the capabilities of LLM, despite the associated costs.* Our logical plan optimizer can yield larger savings (in both runtime and monetary cost) compared to the optimization overhead. Our empirical studies with a case study show that the logical plan optimizer can reduce end-to-end execution time by 70% and processing price by 94%, respectively.

Question 2: Given logical plans for semantic data analytics, do physical plan optimizations exist? Service providers usually offer a variety of LLM options. For instance, OpenAI offers three sizes of its GPT-4.1 series: GPT-4.1, GPT-4.1 mini, and GPT-4.1 nano. These models exhibit heterogeneous performance and costs, where a more powerful LLM is inevitably more expensive. Each semantic operator in a plan can choose a variety of models to use. Among

all LLM options, some are overkill for simple analytics requests, while others may underperform on complex ones. Nirvana defines a physical plan as a choice of execution strategy (*i.e.*, models in this work) for operators in a logical plan, and we apply physical plan optimization through model selection. Jo et al. [10] proposed Smart, a model selection method, which leverages a mix of LLMs to provide similar levels of output quality as the most powerful LLM but at a much lower expense. However, Smart only supports selecting LLMs for a single operator in a fixed task. Nirvana supports general query plans, which optimizes physical plans by assigning the most cost-effective model to each operator based on an improvement score, a novel measure of model cost-effectiveness. To improve optimization efficiency, we introduce two techniques, *computation reuse* and *evaluation pushdown*, to eliminate unnecessary computations. The empirical study (Section 5) finds that these optimizations reduce optimization overhead by 4× compared to Smart, while maintaining system effectiveness.

In summary, the key contributions are as follows:

- We discuss differences between semantic data analytics and relational data analytics, motivating an agent-based logical plan optimizer that uses a random walk search framework and an LLM-as-a-judge plan evaluator for logical plan exploration and exploitation.
- We design a physical plan optimizer that selects the most cost-effective LLM backend per operator in a logical plan, minimizing inference costs while preserving result quality.
- The core of the physical plan optimizer is a model selection algorithm based on the novel concept of an improvement score. To reduce optimization overhead, we propose optimization techniques — computation reuse and evaluation pushdown — with a model capability hypothesis.
- Extensive experiments on three real-world multi-modal benchmarks show that Nirvana can reduce end-to-end runtime by 10%–85% and monetary cost by 76%, compared to state-of-the-art systems. Each component plays its unique role in facilitating system’s efficiency and scalability.

The remainder of the paper is organized as follows. Section 2 provides an overview of Nirvana. Section 3 presents the logical plan optimizer in Nirvana. Section 4 presents the physical plan optimizer. We report experimental results in Section 5. We discuss related work in Section 6 before concluding and discussing future directions in Section 7. Nirvana is open-sourced at <https://anonymous.4open.science/r/Nirvana>.

2 System Overview

We first provide background on the data model and analytical queries supported by Nirvana. Then, we define the notions of logical and physical plans in the context of Nirvana and outline the optimizations involved.

2.1 Background: Data Model & Operators

Nirvana supports a data model consisting of tables with both structured and unstructured fields. Unstructured fields may contain heterogeneous content such as text, documents, images, or audio. Figure 1 presents an example of a movie dataset containing numeric, textual, and image fields. Internally, Nirvana represents

unstructured fields as text that store file paths pointing to remote locations (*e.g.*, web URLs or object storage such as S3 [1]) where the unstructured data resides.

To support flexible and versatile semantic data analytics, Nirvana offers a suite of semantic operators. Table 1 summarizes the operators currently implemented in Nirvana: The map operator applies an LLM-driven transformation to a target column, producing a new derived column. The filter operator retains records based on a natural language predicate. The reduce operator applies a many-to-one aggregation across multiple data in a target column using a user-specified reducer expressed in natural language (NL). The rank operator assigns rankings to input values using an NL-defined ranking criterion. While Table 1 lists the currently supported operators, Nirvana is extensible with support for additional operators such as join and group by in this work. Users can construct their own analytical queries tailored to their data and tasks by employing semantic operators in Nirvana. The following is an example query to analyze common characteristics of crime movies with ratings between 8.5 and 9:

```
1 import nirvana as nv
2 df = nv.DataFrame(movie_data)
3 df.map(user_instruction="According to the movie overview,
4       extract the genre of each movie.", input_column="
5       Overview", output_column="Genre")
6 df.filter(user_instruction="The rating is higher than 7.",
7       input_column="Rating")
8 df.filter(user_instruction="The rating is lower than 8.",
9       input_column="Rating")
10 df.filter(user_instruction="The movie belongs to crime
11      movies.", input_column="Genre")
12 df.reduce(user_instruction="Summarize the common points
13      of their stories.", input_column="Overview")
```

Listing 1: Query example.

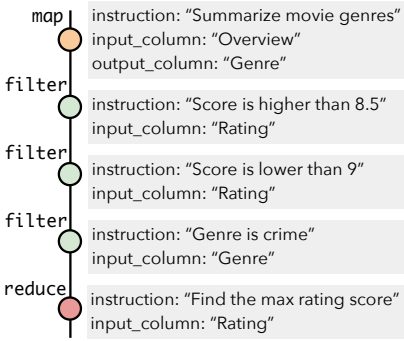
2.2 Query Optimization

Similar to query optimization in relational systems, Nirvana offers system-level query optimization. In Nirvana, a user-specified query is compiled into a data lineage graph (a directed acyclic graph), which serves as a logical plan in this paper. A logical plan captures the execution order of semantic operators, along with their predicates or instructions and their scopes of application, as illustrated on the left side of Figure 2, where a logical plan is derived from the query in Listing 1. A corresponding physical plan specifies the actual execution engines (*e.g.*, computing functions or AI models) used to implement each operator in the logical plan. Next, Nirvana offers logical and physical plan optimizations. Inspired by the Spark SQL system [3], Nirvana separates query optimization into logical plan optimization and physical plan optimization.

While logical plan optimization in Nirvana mirrors and extends traditional rule-based approaches, Nirvana devises a key innovation — an agentic optimizer — and introduce several transformation rules specific to semantic processing (*i.e.*, operator re-ordering, operator fusion, and operator replacement). As indicated in Figure 2, the agentic optimizer explores multiple candidate logical plans and estimates their result quality (measured by accuracy) and execution cost. It then selects the plan that minimizes the cost while meeting a quality constraint. The details of the logical plan optimizer are

Table 1: Semantic operators currently supported in Nirvana.

Operator	Definition	Usage Example
map	Applies an LLM to perform a transformation on a column, producing a new column as output	df.semantic_map(user_instruction="NL-specified transformation", input_column="colA", output_column="colB")
filter	Uses an LLM to evaluate a natural language predicate on a column	df.semantic_filter(user_instruction="NL-specified predicate", input_column="colA")
reduce	Aggregates values in a column based on an NL-specified reduction function	df.semantic_reduce(user_instruction="NL-specified reducer", input_column="colA")
rank	Ranks values in a column according to an NL-specified ranking function	df.semantic_rank(user_instruction="NL-specified ranker", input_column="colA")

Initial Query Plan from an Input Query**Logical Plan Optimization****Plan generation by LLMs**

Here are some operators:
{op_info}
 The available transformation rules:
{rules}
 The given logical plan:
{logical plan}
 Optimize the logical plan by applying given transformation rules...

Answer

The optimized plan:
{optimized logical plan}

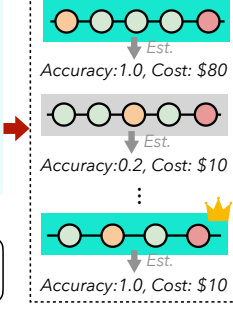
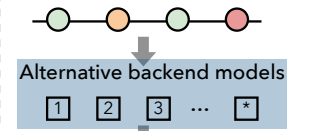
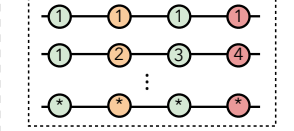
Candidate logical plans**Physical Plan Optimization****Logical plan****Candidate physical plans**

Figure 2: System overview of Nirvana. Given an initial query plan, the logical optimizer samples and rewrites the logical plan to identify a (near-)optimal plan in the logical plan optimization phase. Then, the physical optimizer assigns the most cost-effective model to each operator in the logical plan.

covered in Section 3. As shown to the right part in Figure 2, following the logical plan plan optimization, in Nirvana, the physical plan optimizer assigns appropriate backend engines (*i.e.*, LLMs) to each operator in the selected logical plan. The goal is to choose the most cost-effective configuration under the quality constraint. Details are presented in Section 4.

3 Logical Plan Optimizer

We begin by presenting the logical plan optimizer, which applies transformation rules to the initial (user-specified) plan to produce an optimized logical plan.

3.1 Optimization Workflow

Algorithm 1 outlines the skeleton of the logical plan optimizer, and Figure 3 illustrates the workflow with a running example.

Initialization. Given a user-specified plan as input, the optimizer initializes the set $\mathcal{P}$ of candidate logical plans with an empty set and sets the best logical plan p^* to the input plan p_0 (lines 1–2).

Plan rewrite. In each iteration, a plan is sampled from the candidate set for rewriting (line 4). We adopt a random walk approach [7] to sample a plan from the candidate set. Each plan is assigned a probability such that all candidates have a chance to be sampled. The random walk spans a tree, and each node represents a logical

plan, as shown in Figure 3, where child nodes are generated from their parent node by rewriting plans. To avoid getting stuck in a local optimum, each plan is sampled from a distribution of a mixture of a uniform distribution and a cost-based weighted probability distribution. Specifically, a plan p_i is sample with the probability:

$$Pr(p_i) = \lambda \cdot \frac{1}{|\mathcal{P}|} + (1 - \lambda) \cdot \frac{\exp(cost_{max} - p_i.cost)}{\sum_{p_j \in \mathcal{P}} \exp(cost_{max} - p_j.cost)}, \quad (1)$$

where λ (set to 0.2) controls the tendency to either continue optimizing the currently best observed plan or to explore new branches. This random walk search procedure interpolates between breadth-first search and depth-first search, thereby balancing plan exploration and plan exploitation.

The selected plan is then rewritten by an LLM rewriter using transformation rules expressed in natural language (line 5). The concept of LLM-driven plan rewriting has been validated and adopted in various domains, including in workflow generation [30, 33] and document analytics [22].

Plan evaluation. Once the sampled plan is rewritten, we check for semantic equivalence and evaluate the plan’s cost. If the new plan is semantically equivalent to the input plan and incurs lower cost, it is added to the candidate set (lines 6–8).

Formally verifying semantic equivalence is challenging, especially for semantic queries, with limited prior work. We thus adopt

Algorithm 1: Logical plan optimization

Input: Data samples D_s , an initial plan p_0 , an plan rewriter m , the number of iterations N_{max} , error tolerance ϵ

Output: the optimized logical plan p^*

```

1 Initialize the set of candidate plans  $\mathcal{P} \leftarrow \{p_0\}$ 
2 Initialize the best logical plan  $p^* \leftarrow p_0$ 
3 for  $i \leftarrow 1$  to  $N_{max}$  do
4    $p_i \leftarrow \text{sample}(\mathcal{P})$             $\triangleright$  Sample a plan from candidates
5    $p^+ \leftarrow \text{rewrite}(p_i, m)$ 
6    $p^+.acc, p^+.cost \leftarrow \text{evaluate}(p^+, p_0, D_s)$     $\triangleright$  Estimate accuracy
    and cost of the optimized plan
7   if  $p^+.acc \geq \epsilon \cap p^+.cost \leq p_i.cost$  then
8      $\mathcal{P} \leftarrow \mathcal{P} \cup \{p^+\};$ 
9  $p^* \leftarrow \arg \min_p \{p.cost \mid p \in \mathcal{P}\}$ 
```

a practical definition – *result equivalence*, which is used widely in Text2SQL studies [13, 29]. However, result equivalence can also be vague in semantic analytics, as LLM-executed semantic operators may produce syntactically different but semantically identical outputs for the same inputs. To address this, we use LLM-as-a-judge [31] as an equivalence verifier. We execute both the input plan and the rewritten plan on a data sample. With the outputs from both logical plans, we prompt an LLM to rate the similarity between the two plan outputs on a scale from 0 (completely different) to 10 (exactly same) based on their semantic consistency. The normalized rating score is used as the plan’s accuracy.

For plan cost, the dominant factor, the overall running time and the monetary cost for LLM inference are typically proportional to the number of LLM calls (*i.e.*, the number of data points to process) and the prompt length per data processing. Our cost estimator tracks the number of processed data items and prompt lengths per operator. The total cost is computed as the sum of all operator costs. We assign a selectivity parameter to each operator to monitor changes in the number of data points to be processed throughout the plan. By default, filter has a selectivity of 0.5, reduce has a selectivity of 0, and the other operators have one of 1. We track changes to operators in the plan and adjust their selectivity accordingly. For instance, when multiple operators (*e.g.*, two or three filters) are merged, the new operator’s selectivity is adjusted accordingly (*e.g.*, reduced to half or one third, depending on how many filter operators are fused).

At the end of the optimization, the logical plan optimizer returns the lowest-cost plan that satisfies the accuracy constraint (line 9). In the case of Figure 3, both p_3 and p_4 have low costs, but p_3 is discarded because it fails the semantic equivalence check.

3.2 Transformation Rules

Traditional query optimizers based on frameworks like Volcano [6] and Cascades [5] face several limitations in the context of semantic data analytics:

(1) *Missing optimization opportunities.* These systems lack semantic understanding and cannot exploit opportunities such as merging multiple semantic operators into a single one, generally resulting in sub-optimal logical plans.

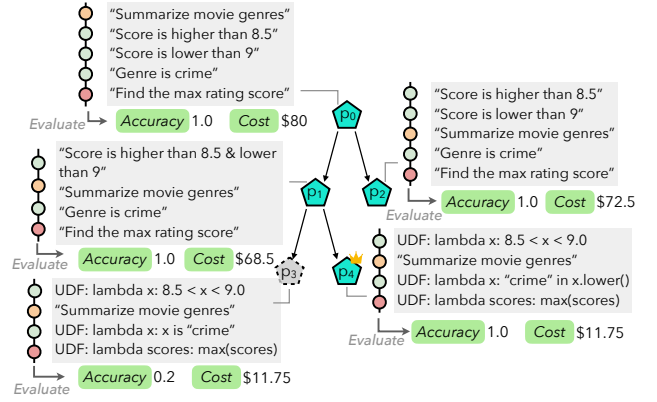


Figure 3: A running example of logical plan optimization where, each node (a logical plan) is sampled by random walk.

(2) *Limited extensibility.* Transformation rules in existing query optimizers are usually hard-coded and tightly coupled with system internals [28]. Integrating a new transformation rule into a commercial or open-source database system is a non-trivial undertaking.

Our agentic logical plan optimizer addresses these issues. First, it leverages the semantic understanding of LLMs to enable more aggressive and intelligent plan rewrites. Second, the transformation rules are expressed in natural language, decoupling them from system internals.

The following three transformation rules are applied in our logical optimizer. Additional rules, formulated in natural language, can be added easily.

- **Filter pushdown.** This transformation, widely used in existing DBMSs, moves a filter operator that does not rely on results of preceding operators to an earlier stage in the plan.
- **Operator fusion.** This transformation merges multiple operators on the same field into one operator. To maintain semantic equivalence, the predicate (or user instruction) will be rewritten for the new operator. For example, in Figure 3, two filter operations with predicates “Score is higher than 8.5” and “Score is lower than 9” can be merged into a single filter with predicate “Score is higher than 8.5 and lower than 9”, yielding fewer LLM calls in processing a query.
- **Non-LLM replacement.** This transformation replaces an operator’s natural language instruction with an equivalent compute function. For example, in Figure 3, “Score is higher than 8.5 and lower than 9” can be interpreted as a comparison function: $8.5 < \text{score} < 9$.

3.3 Local Rewrite Model

The cost overhead is one of the biggest obstacles when using LLMs for plan optimization, as each LLM call is expensive in both latency and price. Especially when using LLM services provided by cloud vendors, most of the time is wasted on network communication. One means of mitigating this problem is to use a local rewrite model as a substitute.

Algorithm 2: Naive backend model selection

Input: Data samples D_s , an operator o , a model set $\mathcal{M} = \{m_1, m_2, m_3, m^*\}$, an improvement margin ΔI_{min}

Output: the most cost-effective model m_o

- 1 Initialize improvement scores $I_{last} \leftarrow 0, I_{curr} \leftarrow 0$
- 2 Initialize the most cost-effective model $m_o \leftarrow m_1$
- 3 $R_{m_1}, R_{m^*} \leftarrow \{m_1(x) \mid x \in D_s\}, \{m^*(x) \mid x \in D_s\}$ $\triangleright$ Results of running m_1 and m^* on D_s used for computing the model cost-effectiveness estimation
- 4 **for** m in $\mathcal{M}$ **do**
- 5 **if** $m = m_1$ **then**
- 6 $I_{curr} \leftarrow 0$
- 7 **continue**
- 8 $R_m \leftarrow \{m(x) \mid x \in D_s\}$
- 9 $I_{curr} \leftarrow$ estimate model cost-effectiveness by computing the improvement score $I_{m_1 \rightarrow m}(D_s, m^*)$ on R_{m_1}, R_m , and R_{m^*}
- 10 **if** $I_{curr} - I_{last} \geq \Delta I_{min}$ **then**
- 11 $m_o \leftarrow m$
- 12 $I_{last} \leftarrow I_{curr}$

Training data collection. A dataset to be used for training a local rewrite model consists of pairs of un-optimized and rewritten logical plans. To assemble such pairs, we first use an LLM to convert natural language data analytics questions into analytical queries using the semantic operators in Nirvana, providing the question and operator interface documentation. Then, we use the LLM with transformation rules to rewrite the logical plan compiled from the generated queries in a last step. Collecting the training set, we fine-tune a small LLM (e.g., Qwen-3-8B, LLaMA-3-8B) using LoRA [8] in an offline setting to obtain the local rewriter.

Model inference. In the inference phase, the sampled logical plan is fed to the local rewrite model to generate an optimized version, *i.e.*, replacing the cloud LLM service with the local model in the optimization workflow. As cloud vendors improve LLM services in latency and others features, the local model's inference needs to be carefully implemented to achieve the same effect. For instance, to further accelerate generation, LLM serving systems, such as vLLM [12] and SGLang [32], are recommended for local deployment.

4 Physical Plan Optimizer

In this work, a physical plan captures the assignment to semantic operators of backend models used as execution engines. Different models with varying scales offer different levels of intelligence, speed, and cost. Selecting the most appropriate model for each operator is essential for cost-effective data analytics, which is the goal of physical plan optimization. For readability, we discuss a case with a search space of four models¹, *i.e.*, $\mathcal{M} = \{m_1, m_2, m_3, m^*\}$. These models have the following properties:

Intelligence (from weakest to strongest): $m_1 \prec m_2 \prec m_3 \prec m^*$

Cost (from cheapest to most expensive): $m_1 \succ m_2 \succ m_3 \succ m^*$

¹Why do we choose a four-model setting to discuss? It can cover most use practices, where users may choose among a Tall-sized model (e.g., BERT), a Grande-sized model (e.g., LLaMA3-8B), a Venti-sized model (e.g., LLaMA3-70B), and a Trenta-sized model (e.g., GPT-4.1). The discussion generalizes to cases with more models.

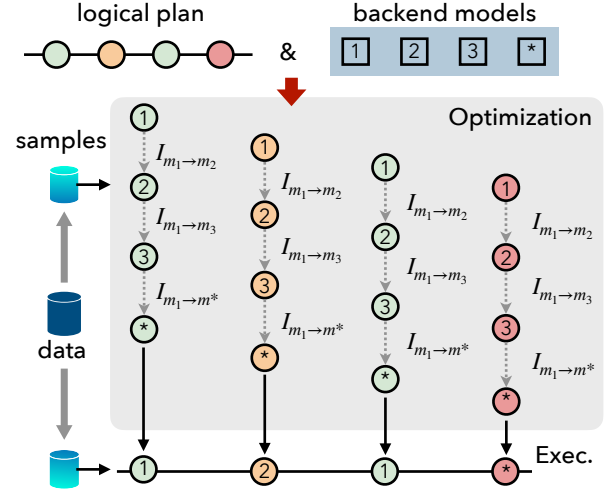


Figure 4: Overview of physical plan optimization.

Model m_1 is the weakest but fastest (and cheapest), while m^* is the most powerful but slowest (and most expensive).

4.1 Optimization Workflow

Given a logical plan with N operators and a choice of M models per operator, the search space of feasible physical plans is of size N^M , yielding a search space that is too large to efficiently find an optimal physical plan. Hence, we constrain the search space by making the following assumption:

HYPOTHESIS 1 (OPERATOR INDEPENDENCE). *Suppose that a model m_a is more intelligent and costly than a model m_b . Then, choosing m_a over m_b as the execution engine for an operator in a plan improves the overall result quality (but at higher cost), regardless of the models allocated to other operators in the plan.*

Based on this hypothesis, we adopt a greedy solution to solve the model allocation problem. Figure 4 provides an overview of the physical plan optimization process. The system selects the most cost-effective model for each operator from a pool of candidates. In the subsequent execution phase, the chosen model is used to process the remaining data. Algorithm 2 captures the model selection procedure for a single operator, presenting a naive baseline that forms the foundation for our optimizer. Given an operator in the logical plan, we evaluate the cost-effectiveness of each candidate model by computing an improvement score $I_{m_1 \rightarrow m}$ over a sampled dataset D_s . This score quantifies the gain in output quality obtained by replacing the baseline model m_1 with a candidate model m as the execution backend. If the score exceeds a user-defined threshold ΔI_{min} , the model is deemed sufficiently beneficial and is assigned to the operator. To estimate model cost-effectiveness, one always needs to run models m_1 , m , and m^* on the entire sampled dataset, which can be expensive. To reduce this cost, we introduce several optimization techniques that decrease the amount of data that must be processed.

In the following, we define the improvement score and present acceleration strategies based on it.

4.2 Improvement Score Computation

Since the ground-truth result of a query is unknown, we treat the output² of the most powerful model m^* as a proxy for the ground truth. We define the improvement score $I_{m_1 \rightarrow m}$ as the performance improvement gained from switching from the backend model m_1 to a stronger model m . The definition is as follows.

$$\begin{aligned} I_{m_1 \rightarrow m_2}(D, m^*) &\triangleq \mathbb{E}_{x \sim D}[m_2(x) = m^*(x), m_1(x) \neq m_2(x)] \\ I_{m_1 \rightarrow m_3}(D, m^*) &\triangleq \mathbb{E}_{x \sim D}[m_3(x) = m^*(x), m_1(x) \neq m_3(x)] \quad (2) \\ I_{m_1 \rightarrow m^*}(D, m^*) &\triangleq \mathbb{E}_{x \sim D}[m_1(x) \neq m^*(x)], \end{aligned}$$

where $m(x)$ denotes the response of the model m to an input x . And, $m_1(x) = m_2(x)$ (resp. $m_1(x) \neq m_2(x)$) denotes that the outputs from the models m_1 and m_2 are semantically equal (resp. different). For binary-output operators like filter, which are required to return True or False, the model outputs can be compared directly. For other ad-hoc queries, such as map that returns arbitrary values, the model outputs are compared using semantic similarity via semantic embeddings generated by a pre-trained language model, like Sentence-BERT [20].

As shown in Eq. 2, an improvement score $I_{m_1 \rightarrow m}$ behaves as an expected proportion of inputs from a dataset D to which the responses of model m are consistent with the surrogate ground truth $m^*(x)$ while responses of the baseline model m_1 are not.

For simplicity, we omit D and x , since models always process the same data with respect to a given operator.

Next, to compute exact improvement scores, it is necessary to invoke the most expensive model m^* on all inputs. To reduce this potentially large overhead, we make the following transformation that reduces the number of m^* invocations.

$$\begin{aligned} I_{m_1 \rightarrow m_2} &\triangleq \mathbb{E}[m_2 = m^*, m_1 \neq m_2] \\ &= \Pr(m_2 = m^*, m_1 \neq m_2) \quad (3) \\ &= \Pr(m_2 = m^* \mid m_1 \neq m_2) \Pr(m_1 \neq m_2) \end{aligned}$$

This transformation allows us to execute expensive evaluations $m_2 = m^*$ on only the subset of records that meet the condition $m_1 \neq m_2$, reducing the number of evaluations involving the expensive model, m^* . We refer to this optimization as “*evaluation pushdown*” because its core idea is similar to “*predicate pushdown*” in DBs.

The improvement score $I_{m_1 \rightarrow m_3}$ can be transformed in a similar way, i.e., $I_{m_1 \rightarrow m_3} = \Pr(m_3 = m^* \mid m_1 \neq m_3) \Pr(m_1 \neq m_3)$. However, we can further reduce the number of invocations of the best model m^* by applying the law of total probability:

$$\begin{aligned} I_{m_1 \rightarrow m_3} &= \Pr(m_3 = m^*, m_1 \neq m_3) \\ &= \Pr(m_3 = m^*, m_1 \neq m_3, m_2 = m_3) + \Pr(m_3 = m^*, m_1 \neq m_3, m_2 \neq m_3) \\ &= I_{m_1 \rightarrow m_2} + \Pr(m_3 = m^*, m_2 \neq m_3, m_1 = m_2) \quad (4) \end{aligned}$$

The last term in Eq. 4 can be expanded as follows.

$$\begin{aligned} \Pr(m_3 = m^*, m_2 \neq m_3, m_1 = m_2) &= \\ \Pr(m_3 = m^* \mid m_2 \neq m_3, m_1 = m_2) \Pr(m_2 \neq m_3 \mid m_1 = m_2) \Pr(m_1 = m_2) \end{aligned}$$

From Eq. 4, the number of m^* invocations in the evaluation of $m_3 = m^*$ can be reduced by the conditions $m_2 \neq m_3$ and $m_1 = m_2$, where $m_1 = m_2$ was already evaluated when computing $I_{m_1 \rightarrow m_2}$.

²we use terms “model output” and “response” interchangeably in this paper.

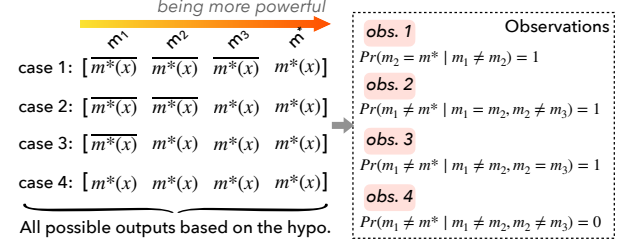


Figure 5: Possible responses of different models to a query, based on the model capability hypothesis. $m^*(x)$ denotes the response of the most powerful model m^* to the input x , while $\overline{m^*(x)}$ denotes the opposite response to the same input.

We also note that the computing $I_{m_1 \rightarrow m_3}$ directly reuses the result of $I_{m_1 \rightarrow m_2}$. Therefore, we call this technique “*computation reuse*”.

These conclusions also apply to the computation of $I_{m_1 \rightarrow m^*}$. For brevity, we only show the expansion of $I_{m_1 \rightarrow m^*}$ using the law of total probability:

$$\begin{aligned} I_{m_1 \rightarrow m^*} &= \Pr(m_1 \neq m^*) \\ &= \Pr(m_1 \neq m^*, m_1 = m_2, m_2 = m_3) + \Pr(m_1 \neq m^*, m_1 = m_2, m_2 \neq m_3) \\ &\quad + \Pr(m_1 \neq m^*, m_1 \neq m_2, m_2 = m_3) + \Pr(m_1 \neq m^*, m_1 \neq m_2, m_2 \neq m_3) \quad (5) \end{aligned}$$

Although *evaluation pushdown* and *computation reuse* help reduce unnecessary computations, considerable computation is still required for evaluations involving the slowest and most expensive model m^* . In the following, we introduce techniques to accelerate this computation.

4.3 Improvement Score Approximation

To achieve acceleration, we first introduce a model capability hypothesis that enables the elimination of slow and expensive m^* invocations when computing improvement scores.

HYPOTHESIS 2 (MODEL CAPABILITY HYPOTHESIS). *Given the ground-truth result y to a query x , if a model m returns a correct answer y to x , i.e., $m(x) = y$, any model m^+ that is more powerful than m also returns the right answer y , i.e., $m^+(x) = y$.*

Although this hypothesis may not hold universally, it is empirically valid in practical cases and can be used to simplify the computation of improvement scores.

Under this hypothesis, we can characterize the possible responses of models in the candidate set as illustrated on the left side of Figure 5, where we treat the output from the best model m^* as the ground truth. In addition, several observations can be made on combinations of model outputs, as shown in Figure 5. For example, $m_2 = m^*$ always holds when $m_1 \neq m_2$ (observation 1 in Figure 5).

Using observation 1, we refine Eq. 3 and obtain the following approximation:

$$\begin{aligned} I_{m_1 \rightarrow m_2} &= \Pr(m_2 = m^* \mid m_1 \neq m_2) \Pr(m_1 \neq m_2) \\ &\approx \Pr(m_1 \neq m_2), \quad (6) \end{aligned}$$

where the evaluations of $m_2 = m^*$ are eliminated completely thanks to the observations based on the hypothesis.

Table 2: Dataset characteristics.

Dataset	#Records	#Attributes	Modalities
Movie	250	22	numerical values, text, images
Estate	1041	4	images, long text
Game	18891	21	dates, numerical values, images, text

The same idea applies to $I_{m_1 \rightarrow m_3}$. From Figure 5, we observe that $m_3 = m^*$ always holds when both $m_2 \neq m_3$ and $m_1 = m_2$ are true. This leads to the following approximation of $I_{m_1 \rightarrow m_3}$:

$$I_{m_1 \rightarrow m_3} \approx I_{m_1 \rightarrow m_2} + Pr(m_2 \neq m_3 \mid m_1 = m_2)Pr(m_1 = m_2) \quad (7)$$

Note that since $Pr(m_1 = m_2) = 1 - I_{m_1 \rightarrow m_2}$, this computation can reuse the result of Eq. 6 so that $Pr(m_2 \neq m_3 \mid m_1 = m_2)$ is the only term to be determined when computing $I_{m_1 \rightarrow m_3}$. Compared to the full evaluation in Eq. 4, the evaluations involving the expensive m^* invocations are entirely eliminated in Eq. 7, and the evaluation of m_3 is constrained to records satisfying $m_1 = m_2$, preserving the benefits of “evaluation pushdown”.

Similarly, we can derive an approximation of $I_{m_1 \rightarrow m^*}$ as follows. This result also follows observations from the hypothesis and is inferred by algebraic transformations (details are provided in Appendix A in the supplementary material).

$$I_{m_1 \rightarrow m^*} \approx Pr(m_1 \neq m^* \mid m_1 = m_2, m_2 = m_3)(1 - I_{m_1 \rightarrow m_3}) + I_{m_1 \rightarrow m_3} - I_{m_1 \rightarrow m_2} + Pr(m_2 = m_3 \mid m_1 \neq m_2)Pr(m_1 \neq m_2) \quad (8)$$

This approximation reduces the number of invocations of m^* considerably. Specifically, evaluations involving m^* are required only on the subset of records that pass the conditions $m_1 = m_2$ and $m_2 = m_3$. Compared to Eq. 5, the approximation reduces the computations by 3× when computing $I_{m_1 \rightarrow m^*}$.

5 Experimental Evaluation

We evaluate Nirvana and several existing proposals on three real-world benchmarks. We also analyze systematically the advantages and limitations of design choices in Nirvana.

5.1 Experimental Setup

5.1.1 Datasets. We use three well-established, publicly available multi-modal datasets for the evaluation; see Table 2. (1) The **Movie**³ dataset includes 250 movie records sourced from OMDB, each with 22 attributes, including titles, directors, ratings, posters, and plot descriptions. It covers numerical, textual, and image modalities. (2) The **Estate**⁴ dataset contains 1,041 estate entries, each with 4 attributes, including property images, project descriptions, locations, and estate details. The modalities include images and long text. (3) The **Game**⁵ dataset contains 18,891 video game records with PEGI ratings from Steam. Each record has 21 attributes, including game titles, cover images, release dates, prices, and game summaries. The modalities span dates, numerical values, images, and long text.

³<https://huggingface.co/datasets/moizmoizmoizmoiz/MovieRatingDB>

⁴<https://huggingface.co/datasets/Binaryy/multimodal-real-estate-search>

⁵<https://github.com/triesonyk/data-analysis-steam-games>

5.1.2 Workloads. For each dataset, we construct a query workload comprising 12 queries. These queries are categorized into three groups based on the number of operations involved: (a) *Small*: Each query contains a single operator. *Small* queries usually process all records in a table, and they are too limited for optimization by traditional optimizers, whereas in Nirvana, they can be further optimized through non-LLM operator replacement and physical plan optimization. (b) *Medium*: Each query consists of two to three operators. *Medium* queries present moderate optimization potential for both traditional systems and Nirvana. (c) *Large*: Each query has four or more operators. *Large* queries provide increased opportunities for optimization, and they usually produce a single-value result, allowing us to evaluate result quality across systems. The workloads are available in Appendix B in the supplementary material.

5.1.3 Systems. We compare Nirvana against three existing data analytics systems:

- GPT-4.1. A baseline that prompts the LLM directly by feeding the query in natural language and entire table into the model to obtain analytical results.
- Palimpzest⁶. An LLM-powered analytical processing system that implements several operators to execute analytical queries. It employs a Cascade optimizer to enable logical and physical plan optimization tailored to semantic operators.
- Lotus⁷. A Pandas-like programming framework with LLM-powered semantic operators, each of which can be executed using a standard algorithm or an approximate algorithm. Optimization in Lotus involves choosing an appropriate executor for each operator, subject to accuracy guarantees.

5.1.4 Implementation. In the experiments, we employ the following OpenAI models as execution backends for semantic operators: gpt-4.1-nano, gpt-4o-mini, gpt-4.1-mini, gpt-4.1. The gpt-4o model is employed for logical plan optimization. Model pricing follows the official OpenAI pricing policy [18]. For logical plan optimization, the number of optimization iterations N_{max} is set to 3. The error tolerance ϵ is set to 80%. For physical plan optimization, the improvement margin ΔI_{min} is set to 20%, and the ratio of data samples is set to 5%. To improve throughput, Nirvana leverages concurrent programming, with the maximum number of coroutines set to 16.

5.2 Overall Results

5.2.1 Settings. We compare Nirvana to two state-of-the-art bulk semantic processing systems, Palimpzest and Lotus, in terms of runtime, monetary cost, and analytical quality. For fair comparison, Palimpzest is configured with access to gpt-4o, gpt-4o-mini, text-embedding-3-small, and clip-ViT-B-32. All built-in implementation rules in Palimpzest are enabled, and its optimization policy is set to MinTime on Movie and Estate and MinCost on Game. For Lotus, we use gpt-4.1 as the backend model and gpt-4.1-nano as the helper model. Both Palimpzest and Lotus are configured to use concurrent programming, with the number of workers set to 16.

⁶<https://github.com/mitdbg/palimpzest/releases/tag/0.7.7>

⁷<https://github.com/lotus-data/lotus/releases/tag/v1.1.0>

Table 3: Runtime and monetary cost of Nirvana, Palimpzest, and Lotus on the Movie, Estate, and Game benchmarks. Mean values across queries are reported with standard deviations.

Dataset	Workload	Palimpzest		Lotus		Nirvana			
		Runtime (s)	Cost (\$)	Runtime (s)	Cost (\$)	Runtime (s)	Cost (\$)	Δ Runtime	Δ Cost
Movie	Small	81.754 $\pm$ 0.194	0.275 $\pm$ 0.137	40.283 $\pm$ 25.278	0.170 $\pm$ 0.147	19.963 $\pm$ 11.298	0.091 $\pm$ 0.160	$\downarrow$ 50.44%	$\downarrow$ 46.48%
	Medium	82.685 $\pm$ 1.371	0.158 $\pm$ 0.004	62.326 $\pm$ 72.450	0.107 $\pm$ 0.096	16.956 $\pm$ 11.686	0.011 $\pm$ 0.004	$\downarrow$ 72.79%	$\downarrow$ 89.50%
	Large	163.115 $\pm$ 61.796	0.487 $\pm$ 0.288	76.178 $\pm$ 8.587	0.383 $\pm$ 0.031	21.169 $\pm$ 5.627	0.014 $\pm$ 0.003	$\downarrow$ 72.21%	$\downarrow$ 96.31%
Estate	Small	363.671 $\pm$ 2.843	1.849 $\pm$ 0.887	149.945 $\pm$ 143.728	0.748 $\pm$ 0.382	114.196 $\pm$ 72.845	0.612 $\pm$ 0.843	$\downarrow$ 23.84%	$\downarrow$ 18.16%
	Medium	581.923 $\pm$ 247.758	2.075 $\pm$ 1.326	252.182 $\pm$ 88.400	1.527 $\pm$ 0.166	193.317 $\pm$ 165.482	0.524 $\pm$ 0.957	$\downarrow$ 23.34%	$\downarrow$ 65.90%
	Large	918.673 $\pm$ 133.471	3.346 $\pm$ 1.131	260.099 $\pm$ 72.787	1.564 $\pm$ 0.541	233.679 $\pm$ 155.603	0.417 $\pm$ 0.265	$\downarrow$ 10.16%	$\downarrow$ 73.37%
Game	Small	$\geq$ 5.5 hours	/	1375.920 $\pm$ 439.464	4.116 $\pm$ 2.852	213.767 $\pm$ 409.620	0.084 $\pm$ 0.154	$\downarrow$ 84.46%	$\downarrow$ 97.96%
	Medium	$\geq$ 5.5 hours	/	2491.667 $\pm$ 980.155	7.419 $\pm$ 4.352	478.463 $\pm$ 540.311	0.187 $\pm$ 0.206	$\downarrow$ 80.80%	$\downarrow$ 97.48%
	Large	$\geq$ 5.5 hours	/	3664.475 $\pm$ 1688.044	10.925 $\pm$ 5.576	728.443 $\pm$ 972.553	0.249 $\pm$ 0.330	$\downarrow$ 80.12%	$\downarrow$ 97.72%

Table 4: Quality evaluation of Nirvana, its competitors, and its variants across the three benchmarks.

	Movie	Estate	Game
GPT-4.1	$\times$	$\times$	$\times$
Palimpzest	58.3%	25.0%	/
Lotus	8.3%	16.7%	25.0%
Nirvana	75.0%	41.7%	50.0%
Nirvana w/o Logical optim.	58.3%	33.3%	41.7%
Nirvana w/o physical optim.	58.3%	33.3%	33.3%
Nirvana w/o all optim.	66.7%	41.7%	41.7%

5.2.2 System runtime. Table 3 presents runtime and monetary cost comparisons across all three datasets. Nirvana consistently outperforms Palimpzest and Lotus in runtime, reducing the execution time by 65.15%, 19.11%, and 81.87% on Movie, Estate, and Game, respectively. Notably, the runtime of Palimpzest on Game exceeds 5.5 hours, making it impractical for real-world deployment. The runtime improvements are attributed to both the logical and the physical optimizations in Nirvana. Compared to Lotus, which supports proxy inference using smaller models, Nirvana additionally reduces LLM computations through operator merging, reordering, and replacement. While Palimpzest also supports operator reordering, Nirvana further enhances performance through collaborative execution between large and small models. We also observe substantial variance in runtime across queries (as reflected in high standard deviations). This variance arises from differences in input data size and instruction complexity across operators, especially in token counts. This highlights an open challenge for future work: estimating semantic processing time and cost prior to execution.

5.2.3 System cost. In terms of monetary cost, Nirvana reduces monetary cost significantly compared to Palimpzest and Lotus. The reasons are two-fold: First, the three transformation rules in our logical plan optimizer effectively reduce LLM invocations. Second, the physical optimizer assigns cheaper, smaller models where appropriate. Since this cost reduction applies at the per-record level, the more data is processed, the larger the savings.

Table 5: Comparison w.r.t. cost of processing query q10 on the Estate dataset between Nirvana’s logical optimizer and the Cascades optimizer in Palimpzest (PZ). Times are in seconds; monetary costs are in USD.

	Opt. time	Opt. cost	Exec. time	Exec. cost
PZ (Cascades)	$\sim$ 0.0	0.0	995.3	2.4
Nirvana	9.8	$8.2E^{-3}$	99.1	0.038

5.2.4 Result quality. Table 4 shows the analytical accuracy of Nirvana, three ablations of it, and the three other systems on the three datasets. The quality is measured as the proportion of queries answered correctly, where the ground truth of each query is labeled manually. The LLM, GPT-4.1, fails to answer any query across the datasets because the table content exceeds the maximum context length. Even on a smaller version of the smallest dataset, Movie, where only 5 out of 22 columns are selected, it fails. Nirvana achieves better accuracy than Palimpzest and Lotus across most datasets. However, all systems struggle on queries involving relatively complex mathematical computations over string-like values, such as calculating the average price of a large group of properties.

5.3 Logical Plan Optimizer Evaluation

5.3.1 Settings. We perform an ablation analysis to evaluate the effectiveness of Nirvana’s logical optimizer. Two variants are considered: (1) Nirvana w/o logical optim. that disables the logical plan optimizer; and (2) Nirvana w/o all optim. that disables both the logical optimizer and physical optimizer.

5.3.2 Runtime and cost. Figures 6, 7, and 8 show runtime and monetary costs for Nirvana and its variants across all queries. Generally speaking, across almost all datasets and workloads, both the logical and physical optimizers independently contribute to performance gains, reducing runtime and cost compared to the variant with no optimizations. Thus, logical and physical plan optimizers play their unique roles in query optimization, reducing both runtime and cost considerably. Runtime and cost differences become more pronounced for larger workloads (e.g., queries q9–q12) and larger datasets (e.g., Game), where optimization opportunities increase.

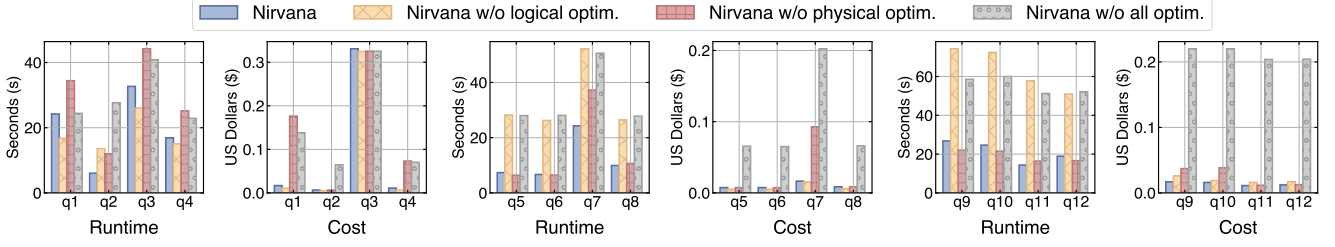


Figure 6: Ablation study of Nirvana on Movie. Runtime and monetary costs are evaluated across all queries.

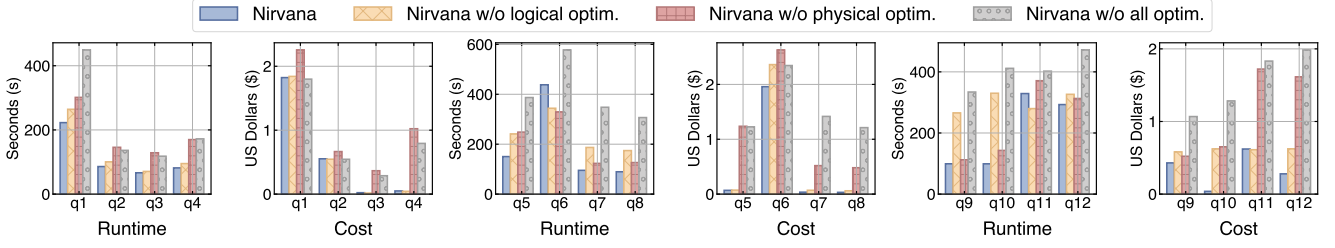


Figure 7: Ablation study of Nirvana on Estate. Runtime and monetary costs are evaluated across all queries.

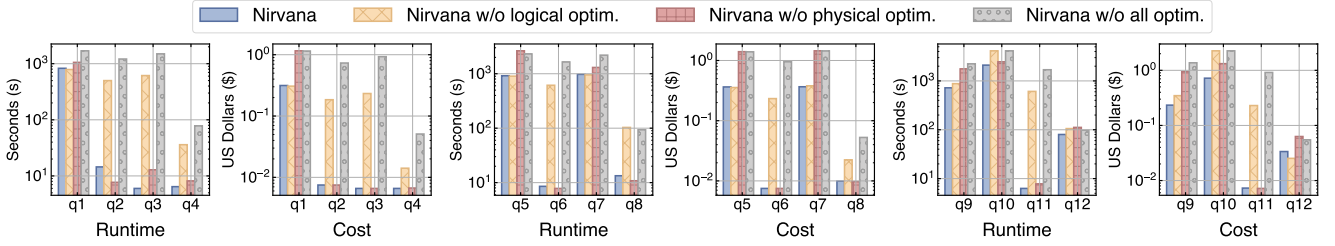


Figure 8: Ablation study of Nirvana on Game. Runtime and monetary costs are evaluated across all queries. The y-axes use a logarithmic scale.

Comparing Nirvana to Nirvana w/o logical optim., we observe that the logical optimizer accelerates query execution and lowers cost notably in most cases, especially for “large” queries (q5–q12), where optimizations like filter pushdown effectively prune irrelevant data early. For smaller workloads (q1–q4), the benefits of logical optimizations are minimal, as these queries usually involve only a single operator. In these cases, the logical plan optimizer in Nirvana has little to do given a single operation, except applying transformation like non-LLM replacement. Indeed, only a few queries — q2 on both Movie and Game — qualify for such replacement. Interestingly, in some cases (e.g., q1 and q3 in Figure 6, and q1 in Figure 8), Nirvana incurs higher runtime and cost than the variant without logical optimization for analytical processing. This is due to the high runtime and cost of the logical plan optimizer that invokes multiple LLMs during the optimization process. We report the overhead of the logical optimizer in Table 5 as a case study. We see that, processing query q10 on Estate requires 9.8 seconds and 0.82 cents for logical optimization alone, compared to the zero-cost, near-instant optimization performed by the Cascades optimizer in Palimpzest. Reducing this overhead presents an important direction for future research on LLM-powered query optimization.

5.3.3 Result quality. We assess the impact of the optimizers on result quality using the results in Table 4. Nirvana with full optimization achieves the same number of correct answers as its no-optimization variant (Nirvana w/o all optim.), while having considerably lower runtime and cost. However, disabling logical optimization (*i.e.*, Nirvana w/o logical optim.) decreases accuracy slightly. This is likely due to a higher likelihood of inference errors when more records are processed by a weaker model, as fewer filters are applied early. And also, an LLM might make more mistakes than a computing function if non-LLM replacement is not enabled. However, the difference is marginal with typically only one or two additional incorrect results out of 12 results if only using either logical or physical plan optimizer.

5.3.4 Case study. Figure 9 illustrates the case of processing q10 on Estate, where the logical plan optimizer contributes to a notable performance gain. We notice that the logical optimizer uses all three transformation rules to rewrite the initial logical plan: (a) Merges three filter operations into one filter and generates a new instruction including all key information from initial instructions; (b) pushes the merged filter operator before the map operation,

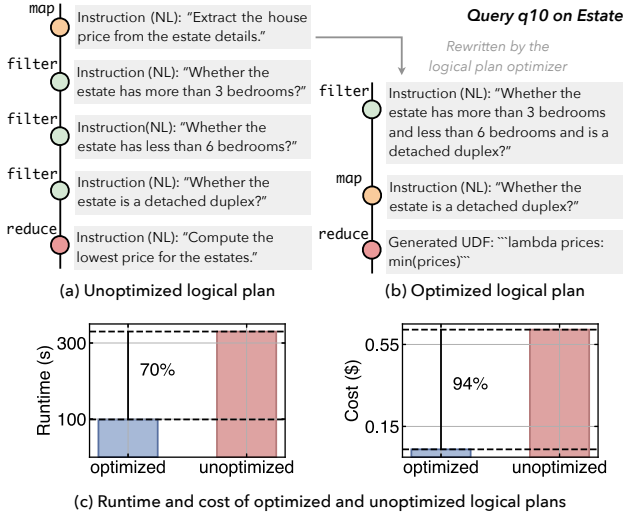


Figure 9: Case study of the runtime and cost of Nirvana with and without logical plan optimization for processing query q10 on Estate.

Table 6: Overhead comparison of query q3 on the Estate dataset between the physical optimizer in Nirvana and the state-of-the-art system, Smart.

	Optim. time (s)	Exec. time (s)	Ratio
Smart (exhaustive)	59.06	626.77	9.42%
Smart (efficient)	53.80	620.95	8.66%
Smart (multi-model)	56.27	625.73	8.99%
Nirvana (Synchronous)	13.11	674.56	1.94%
Nirvana (Asynchronous)	4.12	66.47	5.82%

pruning irrelevant records; (c) and substitutes the LLM-powered reduce operator with a non-LLM reduce implemented by a computing function. With these optimizations, the runtime and cost are reduced by approximately 70% and 94%, respectively.

5.4 Physical Plan Optimizer Evaluation

5.4.1 Settings. We conduct an ablation study of the physical optimizer, comparing Nirvana with two variants: (1) Nirvana w/o physical optim., which disables the physical optimizer, and (2) Nirvana w/o all optim., which disables both optimizers.

5.4.2 Runtime and cost. The physical optimizer provides a robust strategy for reducing runtime and monetary cost. Nirvana w/o physical optim. outperforms the version with no optimization, offering evidence of the physical optimizer’s effectiveness. Further, we observe that Nirvana w/o physical optim. often incurs lower costs than Nirvana w/o logical optim. We conclude that the physical optimizer plays a dominant role in cost reduction. Interestingly, we also find that in certain queries — such as q5 and q6 in Figure 6 — logical plan optimization alone yields larger runtime and cost savings than when using also physical plan optimization. Another

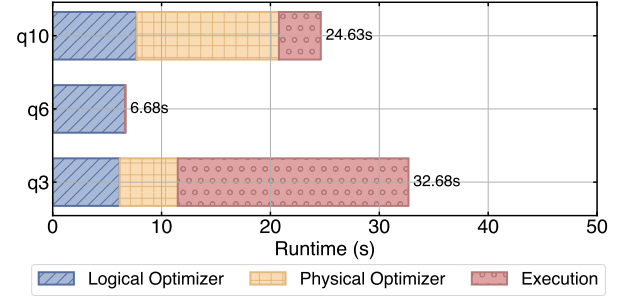


Figure 10: Runtime breakdown of Nirvana on Movie.

interesting finding is that logical optimization can bring more reductions in runtime and cost than physical plan optimization in certain cases where a semantic operator can be replaced by a computing operator (e.g., q5 and q6 in Figure 6). This happens when semantic operators are replaced by traditional computing operators, showcasing that the logical optimizer can sometimes apply more aggressive transformations that yield larger runtime reductions.

5.4.3 Result quality. We evaluate the effect of optimizers on result quality; see Table 4. The findings align with the findings in Section 5.3: the impact of optimization on answer quality is minimal. In some cases, disabling the physical plan optimizer results in one or two additional queries (out of 12) being incorrectly answered.

5.4.4 Comparison with Smart [10]. Model selection is at the core of physical plan optimization. Smart [10] is a recent system that proposes a cost-aware model selection algorithm with accuracy guarantees. However, it focuses on a single filter operator and executes in a non-parallel fashion. It offers three variants: exhaustive, efficient, and multi-model. We compare Nirvana’s physical optimizer in both its non-parallel (Synchronous) and parallel (Asynchronous) modes against all three Smart variants where the candidate LLMs are same as those available to Nirvana. Results are shown in Table 6. First, in the synchronous setting, Nirvana and Smart yield comparable execution times, suggesting that both systems make similar model selection decisions during query planning (i.e., assigning GPT-4.1-nano to all operators as the execution backend by both Smart and our physical optimizer). However, Nirvana’s optimizer requires significantly less time for optimization — about one-fourth that of Smart. In the asynchronous setting, Nirvana drastically reduces both optimization and execution time, showing that parallel execution can boost throughput.

5.5 Runtime Breakdown Analysis

5.5.1 Settings. We provide detailed analysis of runtimes by selecting one query from each of the *Small*, *Medium*, and *Large* workloads and run them on three datasets.

5.5.2 Results. Figures 10, 11, and 12 present the runtime distribution across the three main components in Nirvana: the logical optimizer, the physical optimizer, and query execution. On Movie, the logical and physical optimizers take a significant portion of the processing time due to the dataset’s small size. The runtime overhead of the logical and physical plan optimizers increases slightly

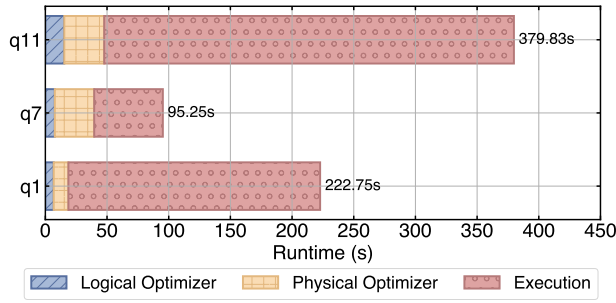


Figure 11: Runtime breakdown of Nirvana on Estate.

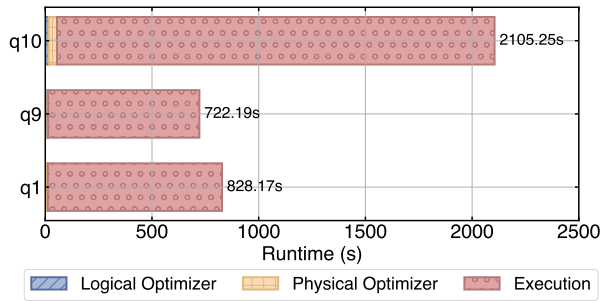


Figure 12: Runtime breakdown of Nirvana on Game.

with the number of operators in a query (from q3 to q10). An exception is q6, which exhibits a different pattern: almost the entire processing time is spent on logical optimization. This is because the logical optimizer replaces all LLM-driven operators with traditional computing functions, eliminating the need for physical optimization and shortening the execution time very substantially. As dataset size increases (from Movie to Estate to Game), the runtime of query execution grows, while the fraction of time spent on query optimization drops sharply. This is expected, as the cost of optimization depends primarily on the number of operators and the size of data samples (which we fix across experiments), rather than the size of the full dataset. These observations highlight an important trend: as workloads scale, the relative overhead of optimization diminishes, making query optimization increasingly valuable. This reinforces the importance of applying both logical and physical optimization to reduce end-to-end runtime, as also evidenced in Figures 6, 7, and 8.

6 Related Work

We proceed to review related work and explain how Nirvana differs from existing approaches.

AI-augmented analytical processing systems. The most relevant line of work involves building semantic data analytical processing systems from scratch, where semantic operators are powered by LLMs [4, 15, 17, 19]. Representative examples include Lotus [19] and Palimpzest [15], which share similar motivations with Nirvana. Lotus adopts an optimization strategy that selects an appropriate execution algorithm for each operator under accuracy constraints. In contrast, Nirvana supports inter-operator optimizations (*i.e.*, the

logical plan optimization in Nirvana), which Lotus does not consider. Palimpzest introduces the Abacus optimizer [21], inspired by the Cascades optimizer [5] from relational systems. However, its plan cost estimation and analysis quality evaluation remain lake refinement. For example, Palimpzest measures analytical quality via benchmark scores, which do not reflect the exact quality to a specific query. Nirvana, on the other hand, explores conceptually distinct optimization strategies and we discuss their pros and cons through experimental evaluations.

Another line of research focuses on integrating AI-powered semantic operators into mature relational DBMSs [11, 24, 26]. For example, ThalamusDB [11] adds a semantic filter operator powered by pretrained models for unstructured data (*e.g.*, text, images, audio) into DuckDB. CAESURA [26] augments traditional SQL operators (*e.g.*, joins, aggregations) with semantic operators such as TextQA and VisualQA to support reasoning over text and images. A key challenge is the difficulty of co-optimizing relational and semantic operators within conventional relational architectures.

LLM-based document analytics systems [2, 14, 22] are also relevant. These systems focus on accurate document analytics, prioritizing correctness over efficiency, whereas Nirvana targets efficient, cost-aware analytics over multi-modal structured datasets.

Workflow generation by LLMs. Workflow generation for reasoning tasks via LLMs is conceptually related to logical plan optimization. Aflow [30] samples reasoning workflows from scratch to achieve high accuracy on tasks such as math and commonsense QA. ADAS [9] treats workflow generation as code generation and leverages LLMs to generate reasoning programs. GPTSwarm [33] models workflow generation as computation graph construction and proposes a graph generation strategy. Unlike these systems, Nirvana focuses on optimizing logical plans for semantic data analytical processing. It draws from traditional database principles such as rule-based transformations and cost-based optimization, while balancing analysis quality and system cost.

Optimizations for compound AI systems. Several recent efforts aim to optimize compound AI systems using techniques such as model selection and query reordering. QuestCache [16] reduces LLM invocation costs for semantic data analytics by reordering table rows and columns to improve KV-cache reuse. Smart [10] reduces the cost of semantic processing by selectively replacing powerful LLMs with smaller, cheaper models while maintaining similar output quality. While sharing a similar goal, Nirvana’s physical plan optimizer adopts a distinct strategy. It introduces a novel cost-effectiveness metric and incorporates optimization techniques to reduce both runtime and monetary cost.

7 Conclusion

With the goal of enabling semantic analytical processing over multi-modal data, we present Nirvana, a new LLM-powered semantic data analytics system equipped with semantic operators. To minimize system costs, Nirvana incorporates two novel query optimization techniques: an agentic logical plan optimizer based on random walk search, and a physical plan optimizer that allocates cost-effective models to operators in a query plan. Extensive experiments show that Nirvana is able to consistently outperform existing semantic data analytics systems in terms of both runtime and monetary cost.

References

- [1] Amazon. 2025. S3: Cloud Object Storage. <https://aws.amazon.com/s3/>.
- [2] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Benjamin Sowell, Dan Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. *CoRR* abs/2409.00847 (2024).
- [3] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *SIGMOD*, Timos K. Sellis, Susan B. Davidson, and Zachary G. Ives (Eds.), 1383–1394.
- [4] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding Language Models in Symbolic Languages. In *ICLR*. <https://openreview.net/forum?id=IH1PV42cbF>
- [5] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29.
- [6] Goetz Graefe and William J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. 209–218.
- [7] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *SIGKDD*. 855–864.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- [9] Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated Design of Agentic Systems. In *ICLR*. <https://openreview.net/forum?id=t9U3LW7JVX>
- [10] Saehan Jo and Immanuel Trummer. 2024. SMART: Automatically Scaling Down Language Models with Accuracy Guarantees for Reduced Processing Fees. *CoRR* abs/2403.13835 (2024).
- [11] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3 (2024), 186.
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*. 611–626.
- [13] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Proc. VLDB Endow.* 17, 11 (2024), 3318–3331.
- [14] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeighami, Aditya G. Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *CoRR* abs/2405.04674 (2024).
- [15] Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J. Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *CIDR*.
- [16] Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Gaspar Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E. Gonzalez, and Matei Zaharia. 2025. Optimizing LLM Queries in Relational Data Analytics Workloads. In *MLSys*.
- [17] Samuel Madden, Michael J. Cafarella, Michael J. Franklin, and Tim Kraska. 2024. Databases Unbound: Querying All of the World's Bytes with AI. *Proc. VLDB Endow.* 17, 12 (2024), 4546–4554.
- [18] OpenAI. 2025. Pricing policy. <https://openai.com/api/pricing/>.
- [19] Liana Patel, Siddharth Jha, Parth Asawa, Melissa Pan, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Analytics Over Text Data. *CoRR* abs/2407.11418 (2024).
- [20] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*. 3980–3990.
- [21] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *CoRR* abs/2505.14661 (2025).
- [22] Shreya Shankar, Aditya G. Parameswaran, and Eugene Wu. 2024. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *CoRR* abs/2410.12189 (2024).
- [23] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *NeurIPS*. http://papers.nips.cc/paper_files/paper/2023/hash/77c33e6a367922d003ff102ffb92b658-Abstract-Conference.html
- [24] MindsDB Team. 2025. MindsDB: AI's query engine. <https://mindsdb.com/>.
- [25] TPC-H. 2025. TPC-H V3 Top Performance Results, 30-Jun-2025 at 9:01 AM. https://www.tpc.org/tpch/results/tpch_perf_results5.asp?resulttype=all&version=3.
- [26] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *CIDR*. <https://www.cidrdb.org/cidr2024/papers/p14-urban.pdf>
- [27] Jiayi Wang, Guoliang Li, and Jianhua Feng. 2025. iDataLake: An LLM-Powered Analytics System on Data Lakes. *IEEE Data Eng. Bull.* 49, 1 (2025), 57–69.
- [28] Yifan Wang, Haodi Ma, and Daisy Zhe Wang. 2024. No more optimization rules: LLM-enabled policy-based multi-modal query optimizer. *CoRR* abs/2403.13597 (2024).
- [29] Chao Zhang, Yuren Mao, Yijiang Fan, Yu Mi, Yunjun Gao, Lu Chen, Dongfan Luo, and Jinshu Lin. 2024. FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis. In *SIGMOD*. 93–105.
- [30] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *ICLR*. <https://openreview.net/forum?id=z5uVAKwmjf>
- [31] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*. http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html
- [32] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*. http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
- [33] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTswarm: Language Agents as Optimizable Graphs. In *ICML*. <https://openreview.net/forum?id=uTC9AFxiIhg>